# Roadwatch: A Geospatial Dashboard for Municipal Road Defect Monitoring in Chennai

**Sidak Singh**
School of Computer Science and Engineering, Vellore Institute of Technology, Chennai, India
*Corresponding author: Sidak Singh | Email: [add institutional email before submission]*

**ABSTRACT—**Urban road defects require timely reporting, prioritization, and maintenance coordination, yet their records may be spread across inspection and communication channels. This paper presents Roadwatch, a browser-based geospatial dashboard designed to organize road-defect records for municipal review in Chennai. The current implementation uses HTML, CSS, and JavaScript, with Leaflet for interactive mapping and a CARTO basemap using OpenStreetMap attribution. Each configured record contains an identifier, road and area labels, geographic coordinates, severity, a confidence field, timestamp, status, crew assignment, and dimensions. Shared client-side filtering supports 24-hour, seven-day, and 30-day views, text search, status and severity chips, sorting, map pop-ups, KPI summaries, and a detail drawer with maintenance-status actions. The implementation establishes an operator-facing interface for reviewing and triaging records. It does not currently include a trained computer-vision model, live sensor or GPS ingestion, server-side persistence, or field-validation instrumentation. Accordingly, no detection-accuracy, localization-error, response-time, or repair-efficiency metrics are claimed. The paper describes the implemented interface, presents a functional validation plan, and identifies the engineering steps required to progress toward a connected road-maintenance workflow.

**INDEX TERMS—** geospatial dashboard, municipal maintenance, road defect monitoring, smart cities, web mapping.

## I. INTRODUCTION

Road-surface defects create safety risks, increase vehicle operating costs, and require municipalities to coordinate inspection, prioritization, and repair. Manual reporting and periodic surveys can leave information distributed across departments or communication channels. Computer-vision research has shown that road damage can be detected from vehicle-mounted smartphone images [1], while later datasets broadened coverage across countries and damage classes [2]. These studies address detection and classification; a municipal operator also needs a usable place to review reported defects, compare urgency, and track follow-up. Roadwatch addresses the operator-facing portion of the workflow: making records inspectable and supporting consistent triage, rather than claiming to solve the upstream detection problem.

## II. RELATED WORK

Maeda et al. reported deep-neural-network road-damage classification from smartphone images and released a dataset used in subsequent research [1]. RDD2022 expanded the dataset to 47,420 images and more than 55,000 damage instances from six countries, including India [2]. A recent survey reviews object detectors and the trade-offs among accuracy, computational cost, and inference speed [3]. Separately, patents describe GPS-tagged pothole records, characteristics, severity, mapping, and maintenance reporting [8], [9]. Therefore, a map-and-workflow interface alone should not be considered novel without a rigorous patentability analysis. Roadwatch focuses on organizing records and exposing a concise municipal triage workflow; automated defect inference and live acquisition remain outside the current software boundary.

## III. REQUIREMENTS AND DESIGN OBJECTIVES

The design objectives were: (1) place records on a geographical map; (2) preserve the same filtered collection in the map and table; (3) expose severity and status clearly; (4) make the latest or most urgent records easy to find; and (5) keep the interface deployable as static web assets. Non-functional requirements include responsive layout, visible map attribution, accessible labels and keyboard paths for table selection, and maintainable separation between record data, filtering logic, and render functions. Requirements for authentication, data retention, uptime, auditability, and privacy remain production concerns.

## IV. SYSTEM ARCHITECTURE

Roadwatch is implemented as a browser-based dashboard using HTML, CSS, and vanilla JavaScript. Leaflet 1.9.4 supplies map controls, circle markers, and pop-up interactions; the basemap is provided by CARTO with OpenStreetMap attribution [4], [5]. The dashboard keeps a client-side array of defect records. A record includes an identifier, road, area, latitude, longitude, severity class, confidence value, detection timestamp, status, assigned crew, and dimensions. A time-range function scopes records to 24 hours, seven days, or 30 days. Text search matches the road, area, or identifier. Severity/status chips and table sorting alter the visible list; the map and table are rendered from the same filtered data, reducing inconsistencies between views.

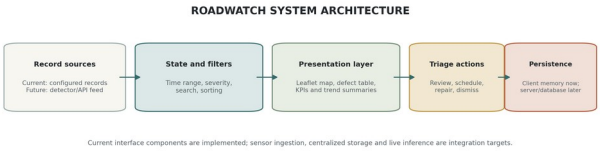



*Fig. 1. Roadwatch architecture; future integrations are identified separately from implemented interface functions.*

### A. Data representation

A defect record is a structured object with an identifier and descriptive fields. Geographic location is represented as latitude and longitude. Severity is one of minor, moderate, severe, or critical; status is one of new, in review, scheduled, repaired, or dismissed. Confidence is displayed as a scalar between 0 and 1. In the current code these values are supplied in the record definition. A production API should validate all fields at the boundary and include provenance, image references, coordinate accuracy, source device, and timestamps with time-zone semantics.

### B. State and filtering

A common state object stores the selected time range, active chip, search query, selected record, and sorting configuration. The date range scopes the collection. The chip and search query refine the rows and map markers. Sorting is applied to the table by severity, confidence, or detection time. The mean-confidence number is the arithmetic mean of the displayed confidence fields within the selected time range; it is not a model calibration measure.

### C. Map integration

Leaflet initializes the map at a Chennai view and displays records as circle markers. Marker radius is visually scaled by severity; repaired records use an outline treatment. Pop-ups summarize the record, and marker selection opens the same record in the detail drawer. The application loads tiles from CARTO and credits OpenStreetMap and CARTO, consistent with the relevant mapping documentation [4], [5]. A tile-provider key must be protected with provider-supported restrictions and usage controls.

## V. IMPLEMENTATION

The source is a single HTML document containing layout styles, the local record array, filtering helpers, render functions, and interaction handlers. The interface includes KPI cards for records needing attention, in-review records, scheduled work, repaired work, and total flagged records. It also includes a critical-defect attention banner, severity counts, a trend display, confidence summary, filter chips, sortable detection table, and a detail drawer. The drawer offers actions for scheduling, marking repaired, dismissing, reopening, and restoring a record. These actions modify in-memory client-side objects; a refresh does not provide the guarantees of server persistence or a transactional audit trail.

## VI. FUNCTIONAL EVALUATION

The current implementation integrates the principal operator-facing functions: a geographical basemap, severity-coded markers, marker pop-ups, a searchable and sortable detection table, time-window controls, KPI cards, a severity summary, a detection-volume view, a mean-confidence display, and a detail drawer. The detail drawer exposes severity, status, configured dimensions, crew assignment, and available status actions. Following the tile-provider and coordinate fixes, the interactive map rendered with visible markers during user-side verification. This is evidence of interface operation only. The current application does not run an embedded computer-vision model, ingest a live camera stream, receive GPS coordinates from a device, or persist updates to a server database. No detection-accuracy, localization-error, response-latency, or repair-outcome metrics are claimed.

| Area | Implementation evidence | Evaluation boundary |
|---|---|---|
| Map | Leaflet map, CARTO tiles, severity markers, pop-ups | Tile availability depends on network/provider |
| Filtering | Search, time windows, status/severity chips | Verify edge cases with recorded test logs |
| Record detail | Drawer with dimensions, confidence and crew fields | Values are record fields; not independently measured |
| Maintenance actions | Schedule, mark repaired, dismiss, reopen/restore | Client-side state; no server audit trail |
| ML / GPS | No live inference or device GPS feed in current source | Accuracy and localization metrics unavailable |

### A. Validation matrix

The appropriate next validation pass should exercise the following cases: (i) initialize the map and confirm tiles and attribution are visible; (ii) verify every marker opens the matching identifier and street; (iii) test all three time ranges against known timestamps; (iv) test empty and non-empty search results; (v) verify status chips and sorting; (vi) execute each status transition and confirm KPI/table/map consistency; (vii) resize to mobile width and change theme; and (viii) reload the page to document the non-persistent state behavior. Record browser version, date, result, and screenshot for each case before submission. No numerical computer-vision performance measures were collected for this interface.

## VII. SMART-CITY RELEVANCE

The system aligns with smart-city operations by making road-defect records easier to locate, compare, and route for maintenance review. Its intended users are municipal road engineers, maintenance coordinators, repair crews, and infrastructure analysts. The concept supports the broad aims of the Smart Cities Mission and is relevant to Sustainable Development Goal 11, particularly safer and more sustainable urban transport [6], [7]. These are intended contributions, not measured citizen-impact outcomes. Operational use would require validated locations, role-based access, persistent storage, audit logs, clear ownership of maintenance decisions, and procedures for verifying or dismissing false reports.

## VIII. LIMITATIONS AND FUTURE WORK

The present implementation is a front-end application with locally configured records and client-side state. Coordinate points are configured along named road corridors; field-captured GPS and geospatial validation are needed before treating locations as maintenance instructions. The confidence, depth, width, severity, timestamp, and crew values in the current record list are stored display fields, not outputs of an integrated inference pipeline or field measurement process. Status changes currently update browser memory and are not durable across sessions. A production design should add an authenticated API, database, event history, image evidence, data-quality checks, duplicate clustering, role-based permissions, and a model-inference service only after those components have been implemented and evaluated. The planned integration sequence is: define an API contract; add a database and server-side validation; connect camera/image processing or an existing road-inspection feed; ingest trusted GPS; implement authentication and role policies; add audit history and notifications; and run a geographically stratified field trial.

## IX. CONCLUSION

Roadwatch shows how a compact geospatial dashboard can organize road-defect records into a shared map, table, summary view, and maintenance-status workflow. Its principal contribution at this stage is interface integration and inspectability. Reliable operational use depends on verified coordinates, a connected and validated defect source, durable storage, security controls, and measured field results. The architecture provides a starting point for those extensions while keeping current evidence distinct from future capabilities.

## REFERENCES


[1] H. Maeda, Y. Sekimoto, T. Seto, T. Kashiyama, and H. Omata, “Road damage detection and classification using deep neural networks with smartphone images,” Computer-Aided Civil and Infrastructure Engineering, vol. 33, no. 12, pp. 1127–1141, 2018, doi: 10.1111/mice.12387.

[2] D. Arya et al., “RDD2022: A multi-national image dataset for automatic road damage detection,” Geoscience Data Journal, vol. 11, no. 4, pp. 846–862, 2024, doi: 10.1002/gdj3.260.

[3] S. H. Abdelwahed et al., “Advancements in real-time road damage detection: a comprehensive survey of methodologies and datasets,” Journal of Real-Time Image Processing, vol. 22, Art. no. 137, 2025, doi: 10.1007/s11554-025-01683-1.

[4] Leaflet contributors, “Leaflet Quick Start Guide,” Leaflet documentation. [Online]. Available: https://leafletjs.com/examples/quick-start/ (accessed Oct. 11, 2026).

[5] OpenStreetMap contributors, “Copyright and License.” [Online]. Available: https://www.openstreetmap.org/copyright (accessed Oct. 11, 2026).

[6] United Nations, “Goal 11: Make cities and human settlements inclusive, safe, resilient and sustainable.” [Online]. Available: https://sdgs.un.org/goals/goal11 (accessed Oct. 11, 2026).

[7] Ministry of Housing and Urban Affairs, Government of India, “Smart Cities Mission Achievements,” Press Information Bureau, Dec. 18, 2024. [Online]. Available: https://www.pib.gov.in/Pressreleaseshare.aspx?PRID=2085711.

[8] “System and method for sensing and managing pothole location and pothole characteristics,” U.S. Patent 9,416,499 B2, Aug. 16, 2016. [Online]. Available: https://patents.google.com/patent/US9416499B2/en.

[9] “Mobile pothole detection system and method,” U.S. Patent 9,365,217 B2, Jun. 14, 2016. [Online]. Available: https://patents.google.com/patent/US9365217B2/en.